15 Implementing an OAuth 2 resource server

This chapter covers

- Implementing a Spring Security OAuth 2 resource server
- Using JWT tokens with custom claims
- Configuring introspection for opaque tokens or revocation
- Implementing more complex scenarios and multitenancy

This chapter discusses securing a backend application in an OAuth 2 system. What we call a resource server in OAuth 2 terminology is simply a backend service. While in chapter 14 you learned how to implement the authorization server responsibility using Spring Security, it's now time to discuss how to use the token the authorization server generates.

In real-world scenarios, you might or might not implement a custom authorization server like we did in chapter 14. Your organization might use a third-party implementation instead of creating custom software. You can find many alternatives out there, ranging from open-source solutions such as Keycloak to enterprise products such as Okta, Cognito, or Azure AD. An example with Keycloak is available in chapter 18 of the book's first edition.

While you have options to configure an authorization server without needing to implement your own, you'll have to implement the authentication and authorization on your backend properly. For that reason, I think this chapter is essential; the skills you learn by reading it have a high probability of helping you with your work. Figure 15.1 reminds you about the OAuth 2 actors and where we are at with our learning plan for this book part.

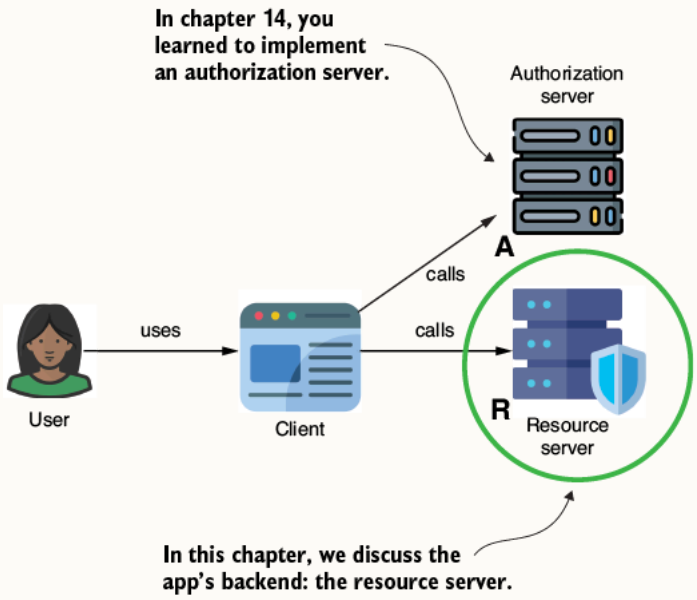


Figure 15.1 In OAuth 2, the app's backend is called a resource server because it protects the users' and clients' resources (data and actions that can be done on data).

We'll begin this chapter in section 15.1 by discussing the resource server configuration for JSON web tokens (JWTs). You'll most often find JWTs used with the OAuth 2 system today; that's why we also start with them. In section 15.2, we discuss customizing JWTs and using the custom values in body or header claims.

In section 15.3, we discuss configuring the resource server to use introspection for token validation. The introspection process is useful when using opaque tokens or when you want your system to be able to revoke tokens before their expiration date.

We'll end the chapter discussing more advanced configuration cases such as multitenancy in section 15.4.

15.1 Configuring JWT validation

In this section, we discuss configuring a resource server to validate and use JWTs, which are non-opaque tokens (they contain data the resource server uses for authorization). To use JWTs, the resource server will need to prove they are authentic, meaning that the ex-

pected authorization server has indeed issued them as a proof of authentication of a user and/or a client. Second, the resource server will need to read the data in the token and use it to implement authorization rules.

We'll learn to configure a resource server by putting it into action, that is, implementing one and configuring it from scratch. We'll start by creating a new Spring Boot project and adding the needed dependencies. We'll then implement a demo endpoint (a resource to use for test purposes) and work on the configuration for authentication and authorization. Here're the steps we'll follow:

1. Add the needed dependencies to the project (in the pom.xml file since we use Maven).
2. Declare a dummy endpoint that we'll use to test our implementation.
3. Implement authentication for JWTs by configuring the service with the public key set URI.
4. Implement the authorization rules.
5. Test the implementation by
   a. Generating a token with the authorization server.
   b. Using the token to call the dummy endpoint we created in step 2.

The following listing presents the needed dependencies. Aside from the web and Spring Security dependencies, we'll also add the resource server starter.

Listing 15.1 Dependencies for implementing a resource server

```
<dependency>                          ①
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

① The resource server starter provides the needed dependencies for implementing an app as an OAuth 2 resource server.

Once we have the dependencies in place, we create a dummy endpoint that we'll use to test our implementation at the end. The following listing presents a simple controller that exposes an endpoint at path /demo.

Listing 15.2 Declaring a simple endpoint for test purposes

```
@RestController
public class DemoController {

  @GetMapping("/demo")        ①
  public String demo() {
    return "Demo";
  }
}
```

① Definition of the dummy endpoint we need to test our configurations after finishing our implementation

For this example, you need to use an authorization server. You can use the one we created in chapter 14 in project ssia-ch14-ex1.

As we want to start both the authorization server and the resource server simultaneously on the same system, we'll need to configure different ports for them. Because the authorization server has the default port 8080, we can change the resource server port to another one. I changed it to 9090, but you can use any free port on your system. The next code snippet shows the property to add to your application.properties file to change the port:

```
server.port=9090
```

Start both the authorization server in project ssia-ch14-ex1 and the current application. You can find this example in project ssia-ch15-ex1 with the projects the book provides.

Remember from chapter 14 that an OpenID Connect authorization server exposes a URL you can use to get its configuration (including the URL for authorization, token, public key set, and others). The next snippet presents the so-called well-known URL:

```
http://localhost:8080/.well-known/openid-configuration
```

You need this link to get information about the URL that the authorization server exposes to provide the public key set that the resource server can use to validate tokens. The resource server needs to call this endpoint and get the set of public keys. Then the resource server uses one of these keys to validate the access token's signature (figure 15.2).

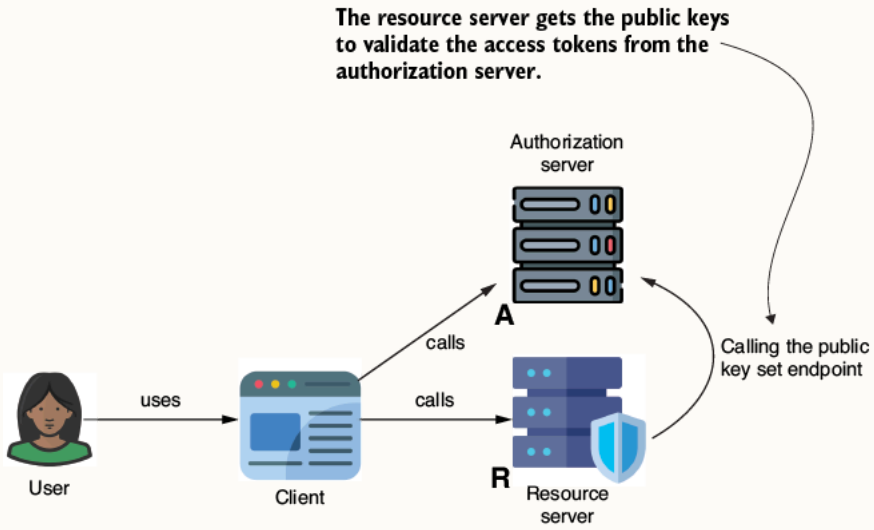


Figure 15.2 The resource server retrieves a set of public keys from the authorization server through an endpoint made available by the authorization server. The resource server then uses these keys to verify the signature of the access token.

Listing 15.3 reminds you about the response you get when calling the well-known configuration endpoint the authorization server exposes. As you can observe, the public key set URI is among the other data provided. The public key set URI is what we need to configure in the resource server so it can validate JWTs.

Listing 15.3 Response of well-known OpenID configuration, containing the key set URI

```
{
  "issuer": "http://localhost:8080",
  "authorization_endpoint": "http://localhost:8080/oauth2/authorize",
  "device_authorization_endpoint":
"http://localhost:8080/oauth2/device_authorization",
  "token_endpoint": "http://localhost:8080/oauth2/token",

  ...

  "jwks_uri": "http://localhost:8080/oauth2/jwks",     ①
  ...

}
```

① The key set endpoint provides the public parts of the asynchronous key pairs configured on the authorization server side. The authorization server uses the private parts to sign the tokens. The resource server can use the public parts to validate them.

To configure the public key set URI, we'll first declare it in the project's application.properties file. The configuration class can inject it into an attribute field and then use it to configure the resource server authentication:

```
keySetURI=http://localhost:8080/oauth2/jwks
```

Listing 15.4 shows the configuration class injecting the public key set URI value into an attribute. The configuration class also defines a bean of the type `SecurityFilterChain`. The application will use the `SecurityFilterChain` bean to configure the authentication, similar to

what we did in book's previous chapters.

Listing 15.4 Injecting the property value in the configuration class

```
@Configuration
public class ProjectConfig {

  @Value("${keySetURI}")        ①
  private String keySetUri;

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    return http.build();
  }
}
```

① Injects the key set URI value in an attribute of the configuration file. You'll need it for the filter chain configuration.

To configure the authentication, we'll use the `oauth2ResourceServer()` method of the `HttpSecurity` object. This method is similar to `httpBasic()` and `formLogin()`, which we used in the second and third parts of this book.

Similar to `httpBasic()` and `formLogin()`, you need to provide an implementation of the `Customizer` interface to configure the authentication. In listing 15.5, you can observe how I used the `jwt()` method of the `Customizer` object to configure JWT authentication. Then I used a `Customizer` on the `jwt()` method to configure the public key set URI (using the `jwkSetUri()` method).

Listing 15.5 Configuring the authentication with JWTs

```
@Configuration
public class ProjectConfig {

  @Value("${keySetURI}")
  private String keySetUri;

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2ResourceServer(            ①
      c -> c.jwt(                         ②
        j -> j.jwkSetUri(keySetUri)       ③
      )
    );

    return http.build();
  }
}
```

① Configuring the app as an OAuth 2 resource server

② Configuring the resource server to use JWTs for authentication

③ Configuring the public key set URL that the resource server will use to validate the tokens

Remember to make the endpoints require authentication. By default, the endpoints aren't protected, so to test the authentication, you first need to ensure your /demo endpoint requires authentication. The following code snippet configures the app's authorization rules. For this example, we can configure all endpoints to require authentication:

```
http.authorizeHttpRequests(
    c -> c.anyRequest().authenticated()
);
```

The following listing presents the full content of the configuration class.

Listing 15.6 The full configuration class

```
@Configuration
public class ProjectConfig {

  @Value("${keySetURI}")
  private String keySetUri;

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2ResourceServer(
      c -> c.jwt(
        j -> j.jwkSetUri(keySetUri)
      )
    );

    http.authorizeHttpRequests(
      c -> c.anyRequest().authenticated()
    );

    return http.build();
  }
}
```

Now you should start the resource server application you just created. Make sure that your authorization server is still up and running. You'll need to use the skills you learned in chapter 14 to generate an access token. Let's remind ourselves of the steps for the authorization code grant type (but remember you could get the token using any other grant type—it's irrelevant to the resource server how you got the access token as long as you have one).

The steps you need to follow with the authorization code grant type are the following (figure 15.3):

1. Redirect the user to log in at the authorization server/authorize endpoint.
2. Use the user's credentials to authenticate. The authorization server will redirect you to the redirect URI and provide the authorization code.
3. Take the authorization code provided after the redirect, and use the /token endpoint to request a new access token.

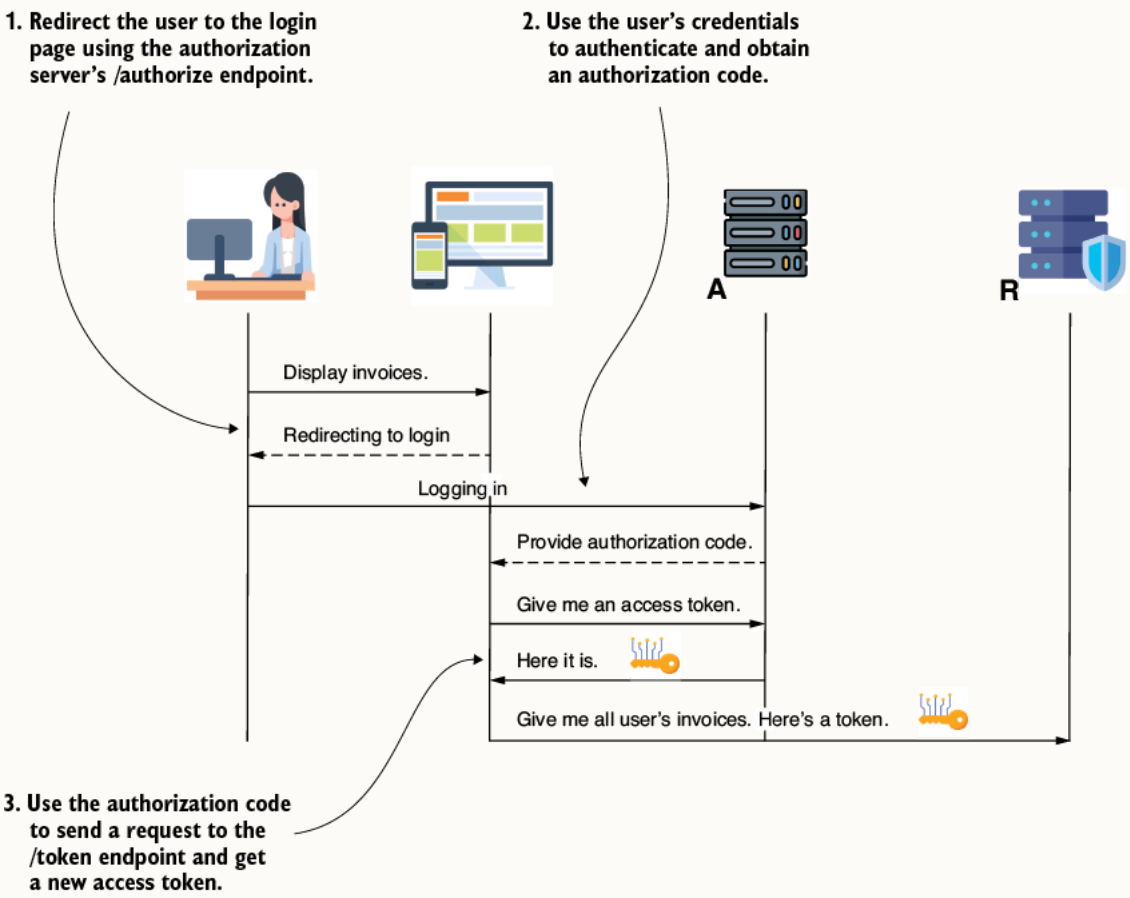


Figure 15.3 The authorization code grant type. The client redirects the user to the authorization server's login page. After the user successfully authenticates, the authorization server redirects back to the client, providing an authorization code. The client uses the authorization code to obtain an access token.

The next snippet (right after the bullet points) shows the URL you can use in your browser to redirect to the authorization server's /authorize endpoint. Remember you need to provide a few parameters, and their values should comply with what you configured in the authorization server. The parameters you must send are

- `response_type` —Use the value `"code"` if you want to use the authorization code grant type.
- `client_id` —The client ID.
- `scope` —The scope you want to access. It can be any of the scopes configured within the authorization server.
- `redirect_uri` —The URI to which the authorization server redirects the client after successful authentication. The redirect URI should be one of those configured in the authorization server.
- `code_challenge` —If using PKCE (proof key for code exchange), you need to provide the code challenge from the code challenge and verifier pair.
- `code_challenge_method` —If using PKCE, you must specify the hash function you used to encrypt the code verifier (for example, SHA-256):

```
http://localhost:8080/oauth2/authorize?response_type=code&client_id=client&scope=openid&redirect_uri=https://www.manning.com/authorized&c
```

NOTE Remember that you must paste the authorization URL in a browser's address bar to send the request.

Log in using the valid user credentials configured in the authorization server, and then wait to be redirected to the requested redirect URI. The authorization server provides the authorization code that you must use in the request to the /token endpoint.

The next snippet shows an example of cURL command sending a request to the /token endpoint to get an access token. Note that I have truncated the authorization code value to fit on the page:

```
curl -X POST 'http://localhost:8080/oauth2/token? \
client_id=client& \
redirect_uri=https://www.manning.com/authorized& \
grant_type=authorization_code& \
code=IhKRpq7GJ7P5VQI_...& \
code_verifier=qPsH3O6-ZDDaOE8DFzVnO5TkN3ZZoVmI_6x4LsVglQI' \
--header 'Authorization: Basic Y2xpZW5OOnNlY3JldA=='
```

The following snippet presents a response body for the /token request. I have also truncated the token values in the snippet:

```
{
   "access_token": "eyJraWQiOiI2Zjk5ZmE3MC...",
   "scope": "openid",
   "id_token": "eyJraWQiOiI2Zjk5ZmE3MC0xNTQ2LTRkMjM...",
   "token_type": "Bearer",
   "expires_in": 299
}
```

You can now use the access token when calling any endpoint that requires authentication. The following snippet shows a cURL command to send a request to the /demo endpoint. Observe that the access token must be sent in the `Authorization` header using the `Bearer` prefix (figure 15.4). The `Bearer` prefix implies that the one having the access token value can use it in the same way as any other party having it.



Figure 15.4 Analogy to the novel *The Lord of the Rings* by J.R.R. Tolkien. The access token is a precious resource. It gives access to several resources to whoever possesses it.

The following snippet shows the cURL command you can use to send a request to the /demo endpoint using the access token from the authorization server:

```
curl 'http://localhost:9090/demo' \
--header 'Authorization: Bearer eyJraW...'
```

15.2 Using customized JWTs

Systems' needs are different from one another, even regarding authentication and authorization. Often, it happens that you need to transfer custom values between the authorization server and the resource server through the access token. The resource server can use such values to apply various authorization rules.

In this section, we'll implement an example where the authorization and resource servers use custom claims in the access token. The authorization server customizes the JWT adding a claim named `"priority"` in the JWT. The resource server reads the `"priority"` claim and adds its value to the authentication instance in the security context. From there, the resource server can use it when implementing any authorization rule.

We'll follow these steps:

1. Change the authorization server to add the custom claim to the access token.
2. Change the resource server to read the custom claim and store it in the security context.
3. Implement an authorization rule that uses the custom claim.

But first things first! We need to add a custom value in the access token's body to the `SecurityConfig` class. In the authorization server, you do this by adding a bean of type `OAuth2TokenCustomizer`. The next code snippet demonstrates such a bean's definition. To simplify things and allow you to focus on the example, I added a dummy value in a field I named `"priority"`. In real-world apps, such custom fields would have a purpose, and you'd potentially have to write certain logic for setting their value:

```
@Bean
public OAuth2TokenCustomizer<JwtEncodingContext> jwtCustomizer() {
```

```
  return context -> {
    JwtClaimsSet.Builder claims = context.getClaims();
    claims.claim("priority", "HIGH");
  };
}
```

With this minimal change, the access tokens now contain a custom `"priority"` field. The next snippet shows a JWT access token I generated in its Base64-encoded format, and listing 15.7 shows the decoded body where you can observe the `"priority"` field:

```
eyJraWQiOiI5ZTBjOTQ5NyOzYmMyLTQ4Y2YtODU5MC04N2JmZjE2ZjczOTAiLCJhbGciOiJSUzI
1NiJ9.eyJzdWIiOiJiaWxsIiwiYXVkIjoiY2xpZW50IiwibmJmIjoxNjg3MjYzMzI5LCJzY29wZ
SI6WyJvcGVuaWQiXSwiaXNzIjoiaHR0cDovL2xvY2FsaG9zdDo4MDgwIiwiZXhwIjoxNjg3MjYz
NjI5LCJwcmlvcml0eSI6IkhJR0giLCJpYXQiOjE2ODcyNjMzMjI9.HrQECSO17tZD8HKXP0U7gm
dmeaO1vPgVypvcf3oR3uawiMdI_joQBsLYOzNWBIgktKn2w9-
rvgtjD2xmhWZgSxRsDW_GZofqOzV9T-
5llMuZlakF7SQLyI67UJZKuPTJK8hBd1OhnurGo7ikPfDWhaqyychKu_uI7SdFrQQVgVqbrmHii
syoURIrI9EwOhBO36M7UPJnIWtOWc34fAoFHxqhPuGIVesHHX5qm6wx-
8_Orjz96eOujVSEuUGRNVtz35_SRjhozcLzgIo3Rt9lUfLI7HSzulfXTCpxtxja-
1E_l_dsk4VHSvLYJUZjlERp5kVJqSO_keaJt8JbDQOnew
```

Listing 15.7 shows the decoded body of the previously presented access token. Remember you can easily use the jwt.io online tool to get the decoded form of a JWT. Alternatively, you can individually Base64-decode either the header or the body of the access token using any other Base64 decoder. The next listing demonstrates our changes on the authorization server work correctly.

Listing 15.7 The Base64-decoded body of the customized JWT access token

```
{
  "sub": "bill",
  "aud": "client",
  "nbf": 1687263329,
  "scope": [
    "openid"
  ],
  "iss": "http://localhost:8080",
  "exp": 1687263629,
  "priority": "HIGH",     ①
  "iat": 1687263329
}
```

① The custom “priority” claim we added to the access token

As a second step, we make the changes on the resource server. You can continue working on the example we used in section 15.1, but to make your learning easier, I created a separate project for this example. You can find the implementation we discuss further in this section in project ssia-ch15-ex2.

The list of steps we need to follow to make our resource server understand the custom claims in the access token is the following:

1. Create a custom authentication object. This object will define the new shape, including the custom data.
2. Create a JWT authentication converter object. This object will define the logic for translating the JWT into the custom authentication object.
3. Configure the JWT authentication converter you created in step 2 to be used by the authentication mechanism.
4. Change the /demo endpoint to return the authentication object from the security context.
5. Test the endpoint, and check that the authentication object contains the custom `"priority"` field.

Listing 15.8 presents the definition of the authentication object. The authentication object should be any class that directly or indirectly inherits the `AbstractAuthenticationToken` class. Since we use JWTs, it’s more comfortable to extend the more specific `JwtAuthenticationToken`. This way, you’ll directly extend the casual shape of an authentication object as designed for JWT access tokens.

Observe that the customization in listing 15.8 adds a field named `"priority"`. This field will hold the value from the custom claim in the access token’s body. In a similar way, you can add any other custom details your app might need for authorization purposes. Having

these details directly in the authentication object from the security context makes configurations easy to write regardless of whether we choose to apply them at the endpoint level (chapters 7 and 8) or method level (chapters 11 and 12).

Listing 15.8 Defining a custom authentication object

```
public class CustomAuthentication
  extends JwtAuthenticationToken {     ①

  private final String priority;       ②

  public CustomAuthentication(
    Jwt jwt,
    Collection<? extends GrantedAuthority> authorities,
    String priority) {

    super(jwt, authorities);
    this.priority = priority;
  }

  public String getPriority() {
    return priority;
  }
}
```

① Customizing the authentication object by extending the JwtAuthenticationToken class

② Adding the custom field "priority"

You have a custom shape of the authentication object, and the next thing you need to do is instruct your app on how to translate the JWT into this custom object. You can do this by configuring a specific `Converter`, as presented in listing 15.9. Observe the two generic types we used: the `Jwt` and the `CustomAuthentication`. The first generic type, the `Jwt`, is the input for the converter, while the second type, the `CustomAuthentication`, is the output. So this converter changes a `Jwt` object (which is a standard contract in Spring Security on how the JWT access token is read) into the custom type we implemented in listing 15.8 (see figure 15.5).

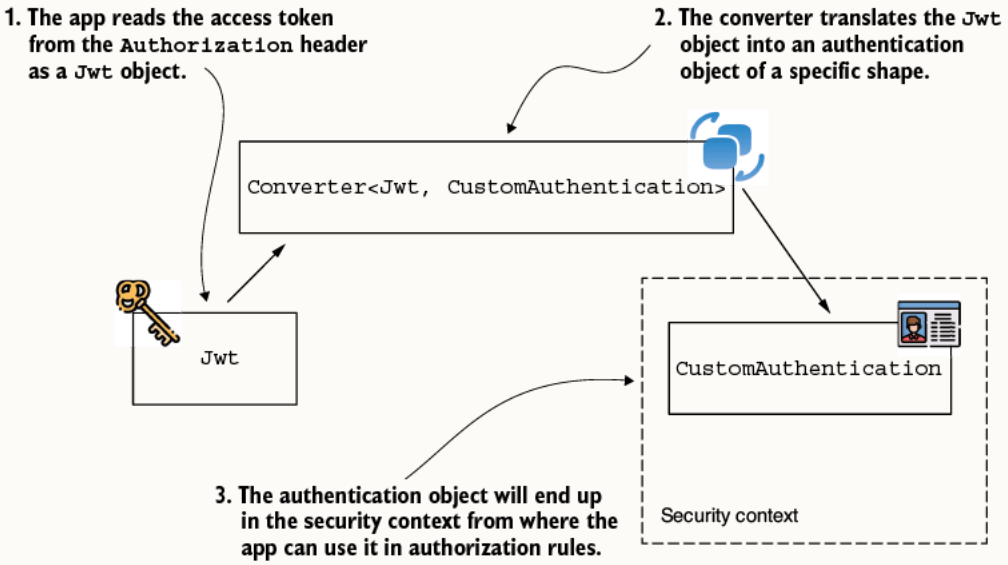


Figure 15.5 A custom converter implements logic to put information from the access token into a custom authentication shape.

Listing 15.9 Converting the access token to an authentication object

```
@Component
public class JwtAuthenticationConverter
  implements Converter<Jwt, CustomAuthentication> {

  @Override
  public CustomAuthentication convert(Jwt source) {
    List<GrantedAuthority> authorities =
      List.of(() -> "read");
```

```
    String priority =                          ①
      String.valueOf(source.getClaims().get("priority"));

    return new CustomAuthentication(source,
                                    authorities,
                                    priority);          ②
  }
}
```

① Getting the priority values from the token's custom claim

② Setting the priority value in the authentication object

You may also observe in listing 15.9 that I have defined a dummy authority. In a real scenario, you'd either take these from the access token (considering they're managed at the authorization server level), or from a database or other third-party system (considering they're managed from a business perspective). In this case, I simplified the example and added a dummy `"read"` authority for all requests. But it's important to remember that this is also the place where you'd deal with authorities (which should also end up in the authentication object from the security context, as they're essential details for authorization rules in most cases).

The next listing shows how to configure the custom converter. In this case, I used dependency injection to get the converter bean from the Spring context. Then I used the `jwtAuthenticationConverter()` method of the JWT authentication configurer.

Listing 15.10 Configuring the custom authentication converter

```
@Configuration
public class ProjectConfig {

  // omitted code

  private final JwtAuthenticationConverter converter;      ①

    // omitted constructor

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2ResourceServer(
      c -> c.jwt(
        j -> j.jwkSetUri(keySetUri)
            .jwtAuthenticationConverter(converter)     ②
        )
    );

    http.authorizeHttpRequests(
      c -> c.anyRequest().authenticated()
    );

    return http.build();
  }
}
```

① Injecting the converter object in a class field

② Configuring the converter object within the authentication mechanism

That's all for the configuration we needed to implement to use the access token's custom claim. Let's test our implementation and prove that it works as expected. The next code snippet shows the changes I made to the /demo endpoint. I made the /demo endpoint return the authentication instance from the security context. Because Spring knows how to inject the value automatically in a parameter of type `Authentication`, I just needed to add this parameter and then make the endpoint's action method return it as is:

```
@GetMapping("/demo")
public Authentication demo(Authentication a) {
```

```
    return a;
  }
```

If everything works as desired, when sending a request to the /demo endpoint, you'll get a response with a body similar to the one presented in the following listing. Observe that the custom `"priority"` attribute correctly appears in the authentication object having the value `"HIGH"`.

Listing 15.11 The /demo endpoint response contains the priority field

```
{
  "authorities": [
    {
      "authority": "read"
    }
  ],
  "details": {
    "remoteAddress": "0:0:0:0:0:0:0:1",
    "sessionId": null
  },
  "authenticated": true,
    ...

  "name": "bill",
  "priority": "HIGH",      ①
}
```

① The custom claim value appears in the authentication instance.

15.3 Configuring token validation through introspection

In this section, we discuss using introspection for access token validation. If your app uses opaque tokens, or if you want a system where you can revoke tokens at the authorization server level, then introspection is the process you must use for token validation. Figure 15.6 will remind you of the introspection process, discussed in detail in section 14.4.

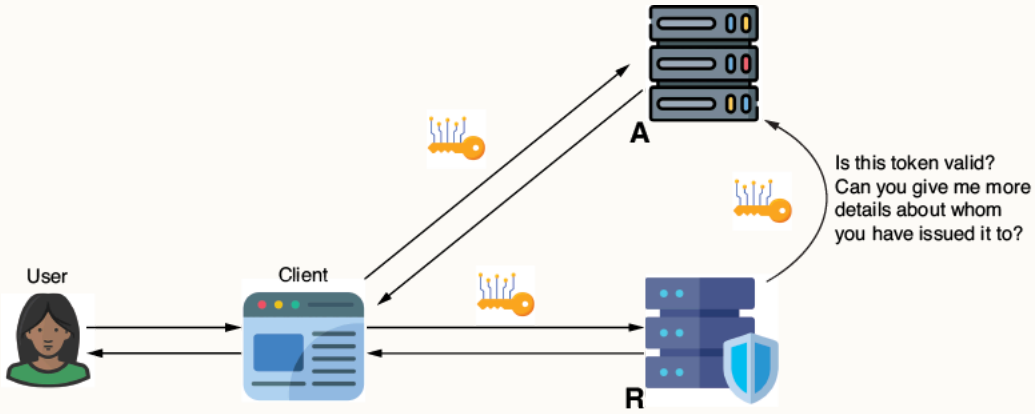


Figure 15.6 Token introspection. In situations where the resource server cannot depend on signature-based validation of access tokens (such as when token revocation is necessary), or when the token does not contain detailed information (as with opaque tokens), the resource server is required to make inquiries to the authorization server. This is done to ascertain the validity of a token and to gather additional information about it.

We'll implement a resource server to demonstrate the use of introspection. To achieve our goal, we must follow these steps:

1. Ensure the authorization server recognizes the resource server as a client. The resource server needs client credentials registered on the authorization server side.
2. Configure authentication on the resource server side to use introspection.
3. Obtain an access token from the authorization server.
4. Use a demo endpoint to prove that the configuration works the way we expect with the access token we got in step 3.

The next code snippet shows you an example of creating a client instance, which we'll register at the authorization server side. This client represents our resource server. As you observe from figure 15.6, the resource server sends requests to the authorization server (for introspection), so this way, it also becomes a client of the authorization server.

To send the introspection requests, the resource server needs client credentials to authenticate, similar to any other client. For this example, I'll change project ssia-ch14-ex4 that we created when discussing opaque tokens in chapter 14:

```
RegisteredClient resourceServer =
  RegisteredClient.withId(UUID.randomUUID().toString())
        .clientId("resource_server")
        .clientSecret("resource_server_secret")
        .clientAuthenticationMethod(
          ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .authorizationGrantType(
          AuthorizationGrantType.CLIENT_CREDENTIALS)
        .build();
```

Remember that passwords and configuration data should never be hardcoded like I did in the previous snippet. I simplified these examples as much as possible to allow you to focus on the subject we're discussing. In a real-world app, you should put the configurations in files outside of the implementation and securely persist secret details (like credentials) somewhere.

The following listing shows how to add both instances of client details (the client's and the resource server's) to the authorization server's `RegisteredClientRepository` component.

Listing 15.12 The `RegisteredClientRepository` definition

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
  RegisteredClient registeredClient =                    ①
    RegisteredClient.withId(UUID.randomUUID().toString())
      .clientId("client")
      .clientSecret("secret")
      .clientAuthenticationMethod(
        ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
      .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
      .tokenSettings(TokenSettings.builder()
          .accessTokenFormat(OAuth2TokenFormat.REFERENCE)
          .accessTokenTimeToLive(Duration.ofHours(12))
          .build())
      .scope("CUSTOM")
      .build();

  RegisteredClient resourceServer =                      ②
    RegisteredClient.withId(UUID.randomUUID().toString())
      .clientId("resource_server")
      .clientSecret("resource_server_secret")
      .clientAuthenticationMethod(
        ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
      .authorizationGrantType(
        AuthorizationGrantType.CLIENT_CREDENTIALS)
      .build();

    return new InMemoryRegisteredClientRepository(
                registeredClient,                        ③
                resourceServer);
  }
```

① Defining a client details instance for the client app

② Defining a client details instance for the resource server (which also becomes a client when calling the introspection endpoint)

③ Adding both client details instances in the authorization server's repository

With the changes made in listing 15.12, we now have a set of credentials our resource server can use to call the introspection endpoint that the authorization server exposes. We can start implementing the resource server. You can find this example in project ssia-ch15-ex3. Listing 15.13 shows how, in the properties files, I configured the three essential values needed for introspection:

- The introspection URI that the authorization server exposes, which allows the resource server to validate tokens

- The resource server client ID that allows the resource server to identify itself when calling the introspection endpoint
- The resource server client secret that the resource server uses together with its client ID to authenticate when sending requests to the introspection endpoint

Along with these, I also changed the server port to 9090, a different one than the application server's (8080), thus allowing both apps to run simultaneously.

Listing 15.13 The resource server application.properties file

```
server.port=9090                          ①
introspectionUri=http://localhost:8080/
  ➥oauth2/introspect                     ②

resourceserver.clientID=resource_server   ③
resourceserver.secret=resource_server_secret   ④
```

① Changing the resource server's port to allow both the resource server and authorization server to run simultaneously

② Configuring the introspection URI as a property

③ Configuring the resource server client ID as a property

④ Configuring the resource server client secret as a property

You can then inject the values in the properties file in fields of the configuration class and use them to set up the authentication. The following listing shows the configuration class injecting the values from the properties file into fields.

Listing 15.14 Injecting the values in fields of the configuration class

```
@Configuration
public class ProjectConfig {

  @Value("${introspectionUri}")          ①
  private String introspectionUri;

  @Value("${resourceserver.clientID}")   ①
  private String resourceServerClientID;

  @Value("${resourceserver.secret}")     ①
  private String resourceServerSecret;

}
```

① Injecting the introspection URI, introspection client ID, and introspection secret from the properties file in fields of the configuration class

Use the introspection URI and the credentials to configure the authentication. You configure the authentication similarly to how we configured it for JWT access tokens—using the `oauth2ResourceServer()` method of the `HttpSecurity` object. However, we call a different configuration method the `oauth2ResourceServer()` customizer object: `opaqueToken()`. For the `opaqueToken()` method, we configure the introspection URI and credentials. The following listing presents this setup.

Listing 15.15 Configuring the resource server authentication for opaque tokens

```
@Configuration
public class ProjectConfig {

  @Value("${introspectionUri}")
  private String introspectionUri;

  @Value("${resourceserver.clientID}")
  private String resourceServerClientID;

  @Value("${resourceserver.secret}")
```

```
  private String resourceServerSecret;

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2ResourceServer(
      c -> c.opaqueToken(                    ①
        o -> o.introspectionUri(introspectionUri)    ②
            .introspectionClientCredentials(      ③
              resourceServerClientID,
              resourceServerSecret)
        )
    );

    return http.build();
  }
}
```

① Configuring resource server authentication for opaque tokens

② Configuring the introspection URI the resource server should use to validate and get details about tokens

③ Configuring the credentials the resource server must use to authenticate when calling the authorization server's introspection URI

Remember to also add the authorization configurations. The next code snippet shows the standard way you learned in chapters 7 and 8 to make all the endpoints require requests' authentication:

```
http.authorizeHttpRequests(
  c -> c.anyRequest().authenticated()
);
```

The following listing shows the full contents of the `configuration` class.

Listing 15.16 Full contents of the configuration class

```
@Configuration
public class ProjectConfig {

  @Value("${introspectionUri}")
  private String introspectionUri;

  @Value("${resourceserver.clientID}")
  private String resourceServerClientID;

  @Value("${resourceserver.secret}")
  private String resourceServerSecret;

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2ResourceServer(
      c -> c.opaqueToken(
        o -> o.introspectionUri(introspectionUri)
            .introspectionClientCredentials(
              resourceServerClientID,
              resourceServerSecret)
        )
    );

    http.authorizeHttpRequests(            ①
      c -> c.anyRequest().authenticated()
    );

    return http.build();
```

```
  }
}
```

① Adding the endpoint authorization configuration. Requests for any endpoint require authentication.

A simple /demo endpoint like the one in the next code snippet is enough for us to test that authentication works correctly:

```
@RestController
public class DemoController {

  @GetMapping("/demo")
  public String demo() {
    return "Demo";
  }
}
```

You can now start both applications: the authorization server and the resource server. Both should run simultaneously. The next code snippet includes the cURL command you can use to send a request to the /token endpoint. To simplify this example, I use the client credentials grant type, but you could use any grant type you learned in chapter 14 to get the access token. Remember that the resource server configuration is the same regardless of how you get the access token:

```
curl -X POST 'http://localhost:8080/oauth2/token? \
client_id=client& \
grant_type=client_credentials' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

If the request is successful, you'll get the access token back in response. The response body looks like the next snippet. I have truncated the token's value to make it fit better on the page:

```
{
   "access_token": "2zLyYA8b6Q54-...",
   "token_type": "Bearer",
   "expires_in": 43199
}
```

In the same way as for the JWT access tokens, when sending a request to a protected endpoint, add the token as the value of the `"Authorization"` header. The access token value must be prefixed with the string `"Bearer"`. The next snippet shows the cURL command you can use to send a request to the /demo endpoint. If everything works correctly, you'll get back the `"Demo"` string in the body in a `200 OK` response status:

```
curl 'http://localhost:9090/demo' \
--header 'Authorization: Bearer 2zLyYA8b6Q54-...'
```

15.4 Implementing multitenant systems

In real-world apps, things aren't always perfect. Sometimes we're in a situation where we must adapt our implementation to match some nonstandard case when integrating with a third party. Also, sometimes we need to implement backends that rely on multiple authorization servers for authentication and authorization (multitenant systems). How should we implement our apps' configurations in such cases?

Fortunately, Spring Security offers flexibility for implementing any scenario. In this section, we discuss implementing resource server configurations for more complex cases, such as multitenant systems or interacting with apps that don't follow the standards.

Let's look at figure 15.7 to revisit Spring Security's authentication design that we discussed in detail in the first two parts of this book. A filter intercepts the HTTP request. The authentication responsibility is then delegated to an authentication manager. The authentication manager further uses an authentication provider, which implements the authentication logic.

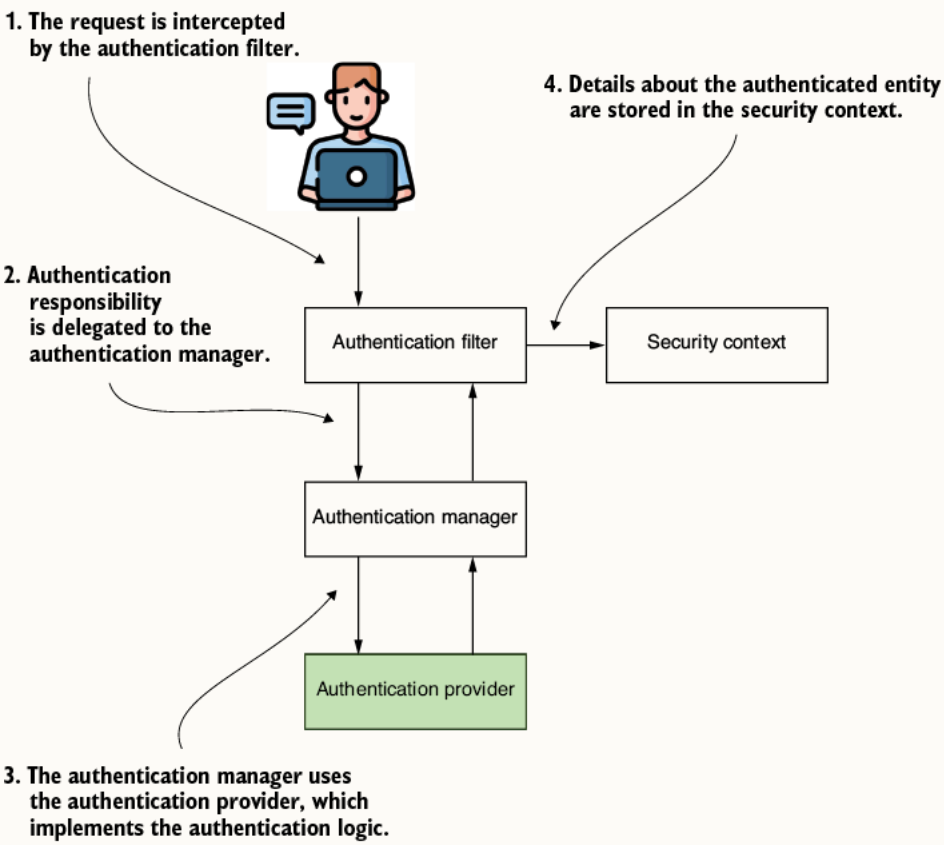


Figure 15.7 The authentication class design. During the authentication procedure, the filter captures the request and passes it on to an authentication manager component. This manager then employs an authentication provider that executes the necessary authentication logic. Upon successful authentication, the application records the authenticated principal's details in the security context.

Why is it important to remember this design? Because for the resource server, like any other authentication approach, you need to change the authentication provider if you want to customize how the authentication works.

In the case of a resource server, Spring Security allows you to plug into the configuration a component named the authentication manager resolver (figure 15.8). This component allows the app execution to decide which authentication manager to call. This way, you can delegate the authentication to any custom authentication manager that can use a custom authentication provider.



Figure 15.8 Implementing an authentication manager resolver, you tell your app which authentication manager to delegate the authentication responsibility to.

If you want your app to use multiple authorization servers all using JWTs, Spring Security even provides an out-of-the-box authentication manager resolver implementation (figure 15.9). For such a case, you only need to plug in the `JwtIssuerAuthenticationManagerResolver` custom implementation that Spring Security provides.

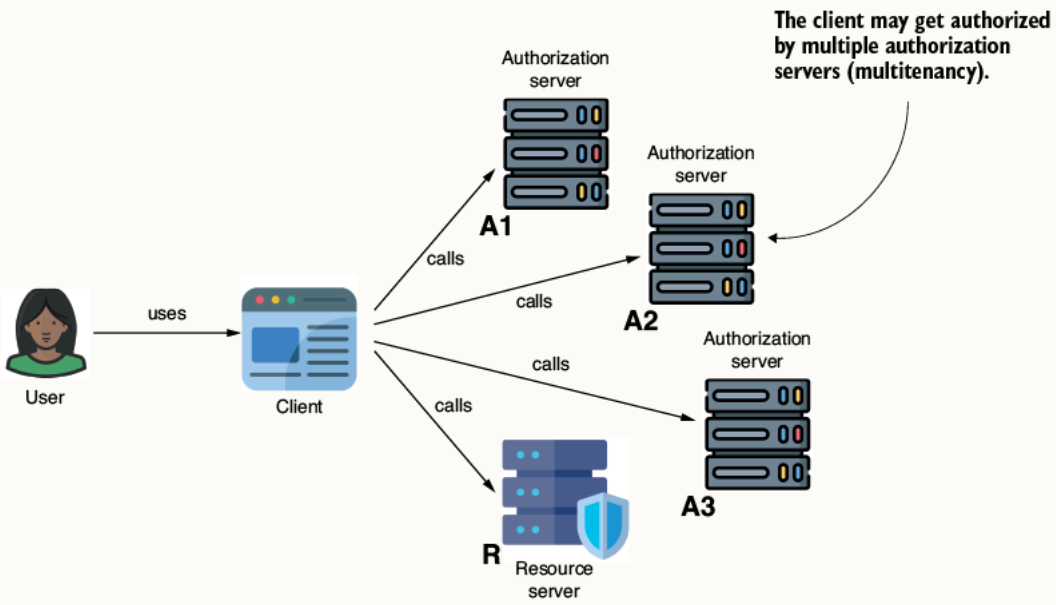


Figure 15.9 Your system might need to use multiple authorization servers to authenticate the users and clients.

Listing 15.17 shows how to use the `authenticationManagerResolver()` method when configuring the authentication. In this example, you observe that I only had to create an instance of the `JwtIssuerAuthenticationResolver` class, for which I have provided all the issue addresses of the authorization servers. You can find this example implemented in project ssia-ch15-ex4.

NOTE Remember to never write URLs (or any similar configurable details) directly in code. We use this approach only with examples to simplify the code and allow you to focus on what's essential for you to learn. Anything customizable should always be written in configuration files or environment variables.

Listing 15.17 Working with two authorization servers that use JWT access tokens

```
@Configuration
public class ProjectConfig {

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.oauth2ResourceServer(
   j -> j.authenticationManagerResolver(
         authenticationManagerResolver())
  );

  http.authorizeHttpRequests(
   c -> c.anyRequest().authenticated()
  );
  return http.build();
 }

 @Bean
 public AuthenticationManagerResolver<HttpServletRequest>
  authenticationManagerResolver() {

  var a = new JwtIssuerAuthenticationManagerResolver(
     "http://localhost:7070",
     "http://localhost:8080");

  return a;
 }
}
```

With a configuration like the one presented in figure 15.10, your resource server works with two authorization servers running on ports 7070 and 8080.

However, sometimes things are more complex. Spring Security cannot provide every customization possible. In such a case, where you need to customize the resource server's capabilities even further, you must implement your custom authorization manager resolver.

Let's consider the following scenario: you need your resource server to work with both JWT and opaque tokens with two different authorization servers. Say your resource server discriminates the requests based on the value of a `"type"` parameter. If the `"type"` parameter's value is `"jwt"`, the resource server must authenticate the request with an authorization server using JWT access tokens; otherwise, it uses an authorization server with opaque access tokens.

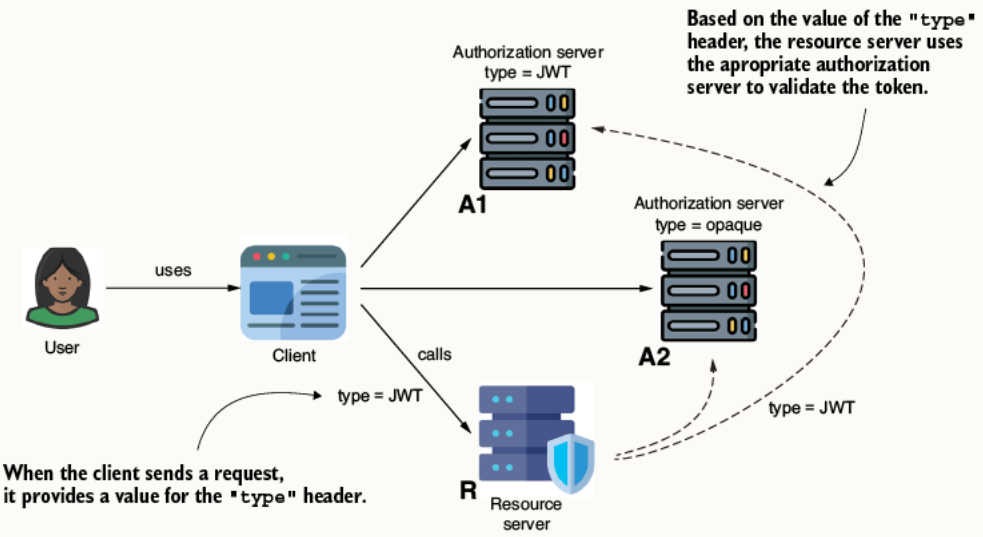


Figure 15.10 Employing two distinct authorization servers, each handling a different type of token. Depending on a specific value in an HTTP request header used by the client, the resource server determines which authorization server to use for validating the access tokens.

Listing 15.18 implements this scenario. The resource server uses a different authorization server based on the value of the `"type"` header in the HTTP request. To achieve this, the resource server uses a different authentication manager based on this header's value.

Listing 15.18 Using both JWT and opaque tokens

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public AuthenticationManagerResolver<HttpServletRequest>
  authenticationManagerResolver(
    JwtDecoder jwtDecoder,
    OpaqueTokenIntrospector opaqueTokenIntrospector
  ) {

  AuthenticationManager jwtAuth = new ProviderManager(          ①
   new JwtAuthenticationProvider(jwtDecoder)
  );

  AuthenticationManager opaqueAuth = new ProviderManager(       ②
   new OpaqueTokenAuthenticationProvider(opaqueTokenIntrospector)
  );

  return (request) -> {                                         ③
   if ("jwt".equals(request.getHeader("type"))) {
     return jwtAuth;
   } else {
     return opaqueAuth;
   }
  };
 }

 @Bean
```

```
  public JwtDecoder jwt'Decoder() {
    return NimbusJwtDecoder                          ④
        .withJwkSetUri("http://localhost:7070/oauth2/jwks")
        .build();
  }

  @Bean
  public OpaqueTokenIntrospector opaqueTokenIntrospector() {      ⑤
    return new SpringOpaqueTokenIntrospector(
      "http://localhost:6060/oauth2/introspect",
      "client", "secret");
  }
}
```

① Defining an authentication manager for the authorization server managing JWT access tokens

② Defining another authentication manager for the authorization server managing opaque tokens

③ Defining the custom authentication manager resolver logic to pick an authentication manager based on the "type" header of the HTTP request

④ Configuring the public key set URI for the authentication manager working with the authorization server that manages JWT access tokens

⑤ Configuring the introspection URI and credentials for the authentication manager working with the authorization server that manages opaque tokens

The following listing shows the rest of the configuration that configures the custom authorization manager resolver using the customizer parameter of the `authenticationManagerResolver()` method.

Listing 15.19 Configuring the `AuthenticationManagerResolver`

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2ResourceServer(
      j -> j.authenticationManagerResolver(      ①
          authenticationManagerResolver(
            jwtDecoder(),
            opaqueTokenIntrospector()
          ))
    );

    http.authorizeHttpRequests(
      c -> c.anyRequest().authenticated()
    );

    return http.build();
  }

  // Omitted code

}
```

① Configuring the custom authentication manager resolver

Even in this example, we used authentication provider implementations offered by Spring Security: `JwtAuthenticationProvider` and `OpaqueTokenAuthenticationProvider`. In this case, `JwtAuthenticationProvider` implements the authentication logic for working with a standard authorization server using JWT access tokens. `OpaqueTokenAuthenticationProvider` implements the authentication logic working with opaque tokens. But you could have even more complex cases in real-world apps.

If you need to implement something very customized, like integrating with a system that doesn't follow any standard, then you can even implement your custom authentication provider.

Summary

- Spring Security offers support for implementing OAuth 2/OpenID Connect resource servers. To configure authentication as an OAuth 2/OpenID Connect resource server, use the `oauth2ResourceServer()` method of the `HttpSecurity` object.
- If you wish to use JWTs, you need to apply the configuration using the `jwt()` method of the `oauth2ResourceServer()` customizer parameter.
- You can also use introspection if your system utilizes opaque tokens or if you want to be able to revoke JWTs at the authorization server side. In such a case, you must configure authentication using the `opaqueToken()` method of the `oauth2ResourceServer()` customizer parameter.
- When using JWTs, you must set up the public key set URI. The public key set URI is a URI exposed by the authorization server. The resource server calls this URI to get the public parts for the key pairs configured on the authorization server side. The authorization server uses the private parts to sign the access tokens, while the resource server needs the public parts to validate them.
- When using introspection, you need to configure the introspection URI. The resource server sends requests to the introspection URI to ask the authorization server whether a token is valid and for more details about it. When calling the introspection URI, the resource server acts as a client for the authorization server, so it needs its own client credentials to authenticate.
- Spring Security offers the opportunity to customize the authentication logic using an authentication manager resolver component. You define and configure such a custom component when you must implement a more specific case, such as multitenancy or adapting your app to a nonstandard implementation.

Previous chapter
< 14 Implementing an OAuth 2 authorization server

Next chapter
16 Implementing an OAuth 2 client >